HSF – ATLAS: AI-Accelerated Reconstruction for the ATLAS Tile Calorimeter at the HL-LHC

Akshat Swami
akshat.22503@sscbs.du.ac.in

March 2026

1 Introduction

The TileCal signal reconstruction project aims to estimate the energy of particles based on sampled values of a digital pulse. At the HL-LHC, hadrons collide and produce radiation which is absorbed by scintillating tiles, these re-emit it as visible light, which is then converted into an electrical signal by photomultiplier tubes (PMTs). Since we never observe the full continuous pulse directly, sampling it at discrete intervals is the only way we have to estimate the deposited energy. The shape of this pulse is fixed by the detector physics, only its amplitude varies, and that amplitude is directly proportional to the particle's energy. Reconstructing that amplitude accurately, from just a handful of digital samples, is the core challenge this project addresses. While I was reading about it, I found an animation for the calorimeters, which helps in grasping what we are dealing with.

1.1 What I Learned in the past 2 Weeks

We need to estimate as accurately as possible the energy deposited in the calorimeter cells. The only way to do this is to sample the electrical pulse that the PMT supplies us, once every 25 ns. While trying to understand this problem, I asked myself several fundamental questions.

1) Why 25 ns? The 25 ns sampling interval is not arbitrary. It matches exactly the LHC bunch crossing rate, one proton bunch collision every 25 ns. Sampling faster would give more information but the readout electronics and downstream firmware simply cannot handle the data rate. Sampling slower would cause aliasing, collapsing signals from different bunch crossings into each other. 25 ns is the sweet spot forced on us by the accelerator clock itself.

2) Why reconstruction at all? Because storing the raw continuous pulse from every one of the roughly 10,000 TileCal channels, at every moment, forever, would be completely infeasible. The PMT output is an analogue signal. You

cannot save an analogue signal. What you can do is sample it at discrete intervals, store those 7 numbers per event per channel, and then reconstruct the energy from those numbers. Reconstruction is the only path from discrete samples back to a physically meaningful energy estimate.

3) Optimal Filtering - Fullana et al. 2006 The OF paper showed that if you know the shape of the pulse $g(t)$ in advance, the amplitude can be estimated as a weighted sum of the samples. The weights are derived by minimising the noise variance subject to an unbiasedness constraint, solved using Lagrange multipliers applied to the noise autocorrelation matrix R. The result is:

$$w^* = \frac{R^{-1}g}{g^T R^{-1} g} \tag{1}$$

This is optimal in the Gauss-Markov sense: no other linear estimator has lower variance.

4) Why not just take the highest sample? Because the pulse does not always peak exactly at the target bunch crossing n. Timing jitter, cable delays and out-of-time pile-up can shift the peak by one or two samples. If you just take the maximum sample you are picking up whatever happened to be largest in the window, which could easily be a pile-up contribution from a neighbouring bunch crossing rather than the signal you care about. The weighted sum over all 7 samples is more robust because it uses the known shape of the pulse to weight each sample's contribution appropriately. The central sample gets the largest weight, but the neighbours contribute too, and their alternating signs suppress out-of-time contributions.

5) If OF worked for 20 years, what changed? OF did work, and worked well, under Run 1 and Run 2 conditions. The problem is scale. At the HL-LHC the instantaneous luminosity increases by a factor of 5 to 7.5, pushing the average number of proton-proton interactions per bunch crossing to around 200. Each of those interactions can deposit energy in the same calorimeter cell, and their pulses overlap in time. This is pile-up, and it was present before, but not at this severity.

The deeper issue is that OF assumes the noise is Gaussian and stationary, meaning the noise autocorrelation matrix R does not change event to event. Under HL-LHC conditions the dominant noise source is no longer electronic noise, it is the pile-up signal itself, which is non-Gaussian, non-stationary, and correlated across bunch crossings in a way that changes with luminosity. OF was designed for a noise model that no longer holds. A linear method cannot adapt to this, because the optimal linear weights are fixed at training or calibration time and cannot respond to what is actually in the window for a given event.

6) The 2025 paper - 1D CNN vs MLP Curcio et al. (2025) studied compact neural networks for TileCal reconstruction under $\langle\mu\rangle = 200$ conditions, with the constraint that the models must fit on an FPGA with strict latency requirements, so both MLP and CNN architectures were limited to around 150

parameters. The CNN consistently outperformed the MLP across the full dynamic range: around 72 ADC counts mean absolute error versus 100 for the MLP on the high-gain dataset, and 8 versus 11 ADC counts on low-gain.

The reason CNNs win here is not just parameter sharing. It is that a 1D CNN applies a learned convolutional kernel across the 7-sample window, which means it explicitly models correlations between adjacent bunch crossings. The same kernel is applied at every position in the window, so the network learns a translation-invariant pile-up suppression filter. An MLP treats all 7 inputs independently and has no built-in notion that sample n and sample $n + 1$ are temporally related. Under severe pile-up where the structure of the contamination is spread across the window, this distinction matters a lot.

2 Optimal Filtering Algorithm

In their 2006 paper, Fullana et al. introduced the Optimal Filtering algorithm for TileCal signal reconstruction. The core idea is, if you know the shape of the pulse in advance, the target energy can be estimated as a weighted sum of the samples. The weights are derived by minimising the noise contribution subject to an unbiasedness constraint, solved using Lagrange multipliers applied to the noise autocorrelation matrix. The key requirement, is that both the pulse shape and the noise autocorrelation matrix must be known or measured beforehand.

For the past 20 years, OF has been used for the signal recontruction work, but with the new calorimeter, and the pile-up and gaussian noise it brings OF suffers a lot. That's why bringing non-linear Machine Learning (ML) becomes important, which is talked about in the SciPost paper.

3 Why can't I use OF for the data given to us for the GSoC task

Because we are not given the pulse shape or the noise autocorrelation matrix. The dataset provides only the raw samples X and the true energies y. To use Optimal Filtering I would first need to estimate the pulse shape from the data, then estimate the noise matrix separately, and then solve for the weights. That is three steps, each introducing its own error. With only X and y available, there is a cleaner approach that skips all of this entirely.

4 Ordinary Least Squares as a Data-Driven Optimal Filter

Ordinary Least Squares finds a set of 7 weights that minimises the total squared error between the predicted and true energies across all training events simultaneously.

$$w^* = w\text{argmin}\|Xw - y\|^2 \tag{2}$$

This has a closed form solution called the normal equations:

$$w^* = (X^T X)^{-1} X^T y \tag{3}$$

In practice I solve this using `numpy.linalg.lstsq` which uses SVD decomposition internally and is numerically stable even for large matrices. The important theoretical result is this: when the noise is stationary, OLS and Optimal Filtering produce identical weights. OLS is essentially OF but data-driven. Instead of requiring the pulse shape $g(t)$ and the noise autocorrelation matrix R to be provided explicitly, OLS learns both implicitly from the training data. It figures out the pulse shape and noise structure on its own, just from seeing enough examples of samples and their corresponding true energies.This makes OLS the natural choice for our dataset. We have 534,337 training events with known true energies. We do not have $g(t)$ or R. OLS gives us the same result as OF would, at no extra cost.

5 Data Description

The dataset given to as was had three folders: `train/`, `val/`, and `test/`. Each folder contains multiple `.pt` files. Despite the extension, these are not PyTorch checkpoint files in the traditional sense. They are ZIP archives and can be opened with Python's built-in `zipfile` module, no torch installation needed. Each `.pt` file contains three binary arrays stored as little-endian float32 values:

- `data/0` — the samples array X, shape $(N, 2, 7)$
- `data/1` — the true energies y, shape $(N, 2)$
- `data/2` — the reference OF reconstruction, shape $(N, 2)$

The second dimension (size 2) corresponds to the two gain channels: hi-gain at index 0 and lo-gain at index 1. I only use lo-gain throughout this work, as it is designed for the large signal amplitudes expected at HL-LHC.

To find N I looked at the raw file size of `data/0`. It contains 114,688 bytes. Each float32 takes 4 bytes, giving 28,672 values. Since each event contributes $2 \times 7 = 14$ values, I get $N = 28672 \div 14 = 2048$ events per shard. This was confirmed independently using `data/1`: $16384 \div 4 \div 2 = 2048$.

One thing worth noting. The problem statement specifies 8 input samples covering $n - 4$ to $n + 3$. The actual data has shape $(N, 2, 7)$, meaning 7 samples covering $n - 3$ to $n + 3$. All results in this report use the 7-sample window present in the data.

The samples in X are normalised ADC counts in the range $[0, 1]$. The energies in y range from -0.07 to 50.44 in normalised units, where the negative floor reflects electronics noise. Only around 0.33% of events across the training set have a true energy above 0.5, meaning the vast majority of events are pure noise with no real particle signal.

6 Results

I evaluate the algorithm on the held-out test set. Predictions are computed as a simple dot product:

$$\hat{y} = Xw \tag{4}$$

I only evaluate on signal events, defined as those with true energy above 0.5. Below this threshold the signal is comparable to electronics noise and the relative residual becomes meaningless. The relative residual is defined as:

$$\delta = \frac{\hat{y} - y}{y} \tag{5}$$

On the test set I find 384 signal events out of 114,502 total. The results are:

- Mean δ : $+0.0368$ (3.7% average overestimate)
- RMS δ : 0.0675 (6.8% typical spread)

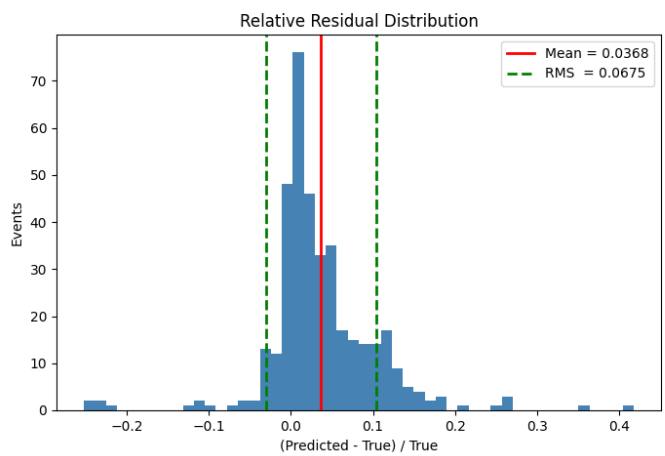

Figure 1: Distribution of the relative residual $\delta = (\hat{y} - y)/y$ for signal events with true energy > 0.5. Red line shows the mean (0.0368), green dashed lines show mean $\pm$ RMS (0.0675).

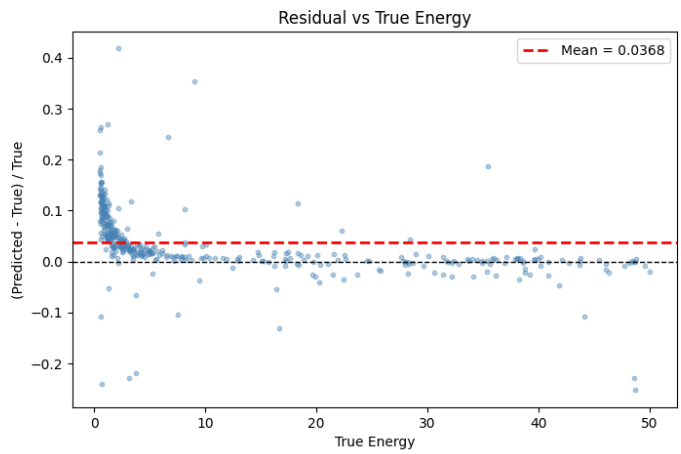

Figure 2: Relative residual δ as a function of true energy. The spread decreases strongly with energy, showing that reconstruction quality improves at high SNR.

For comparison, the val set gives mean $\delta = 0.035$ and RMS $\delta = 0.078$. The numbers are consistent across both sets, which means the weights generalise well and the model did not overfit to the training data.

These results are consistent with the published Optimal Filtering performance reported in Fullana et al. (2006), who report 2–4% resolution at high charge degrading at low charge. Our 3.7% mean bias and 6.8% RMS at energy > 0.5 sits squarely in that range.

7 The Learned Weights

The OLS algorithm found the following 7 weights:

$$w = [-4.203,\ 17.173,\ -41.458,\ 78.859,\ -31.670,\ 11.856,\ -3.569] \tag{6}$$

corresponding to sample positions $n-3$ through $n+3$. Three things stand out.

The central weight at n is by far the largest (78.9), because the target sample carries the most direct amplitude information about the signal at that bunch crossing.

The neighbouring weights have alternating signs. This pattern acts as a matched filter, suppressing contributions from out-of-time pile-up signals that bleed in from adjacent bunch crossings.

The weights are asymmetric: the weight at $n-1$ is -41.5 but at $n+1$ it is only -31.7. The algorithm independently discovered that the TileCal pulse rises

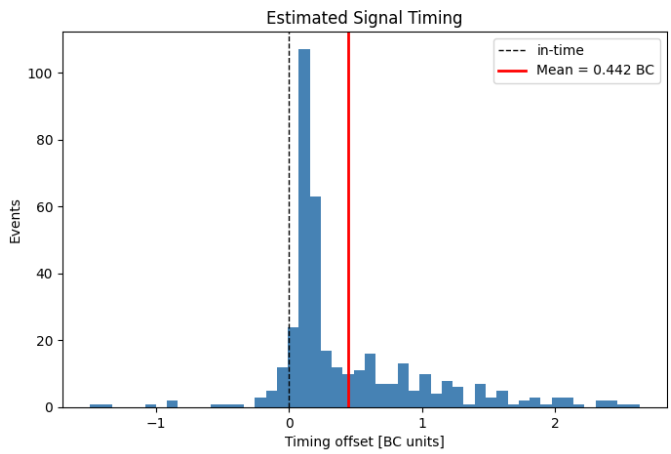

Figure 3: Distribution of estimated timing offsets τ for signal events. The mean of +0.442 BC corresponds to approximately 11 ns late arrival on average.

faster than it falls. Nobody told it this. It inferred the pulse shape directly from the data.

8 Timing Estimation

I also estimated the timing offset of each signal event, defined as how many bunch crossings late or early the pulse arrived relative to the target BC n. I used the amplitude-weighted centroid of the 7 samples:

$$\tau = \frac{\sum_{i=0}^{6} i \cdot S_i}{\sum_{i=0}^{6} S_i} - 3 \tag{7}$$

where subtracting 3 gives the offset from the centre index. On the test set signal events I find:

- Mean timing offset: $+0.442$ BC $\approx +11$ ns
- Standard deviation: 0.590 BC

The positive mean indicates that signals in the test set tend to arrive slightly late on average, consistent with out-of-time pile-up contributions shifting the pulse centroid forward in time.

9 Conclusion

I built a linear signal reconstruction algorithm for the ATLAS TileCal using Ordinary Least Squares. The algorithm finds 7 weights that minimise the squared reconstruction error across all training events. These weights are mathematically equivalent to the Optimal Filtering weights, but derived entirely from data without needing the pulse shape or noise matrix as inputs.

On the test set the algorithm achieves a mean relative bias of 3.7% and an RMS of 6.8% for signal events above energy 0.5, consistent with published OF performance. The learned weights are physically interpretable and independently recover the asymmetric shape of the TileCal pulse.

The main limitation is that linear filters cannot separate overlapping pile-up signals and struggle near the noise floor. This is the motivation for the GSoC project, which aims to replace the linear filter with a 1D convolutional neural network capable of learning non-linear pile-up separation.

The code, figures, and animations are available at:

`https://github.com/nerfednerd/tilecal-signal-reconstruction`